

```
STATUS

ACCOUNT

STATUS

class Banner
  attr_accessible :horiz, :link, :visible, :image, :position
  has_attached_file :image, styles: { vert: '220'

  before_create :assign_position

  def assign_position
    max = Banner.maximum(:position)
    self.position = max ? max + 1 : 0
  end
end
```

FULL STACK DEVELOPMENT

NEXT.JS

AULA 02

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	9
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Nesta aula, você aprenderá como funciona a estilização em aplicações Next, alternativas de estilização e vantagens e desvantagens do uso de algumas delas. Além disso, colocaremos em prática o CSS puro e o uso do tailwind como ferramenta de estilização, aprendendo também como configurá-lo!



HANDS ON

Aprenderemos sobre as possibilidades de estilização dentro do Next, veremos a facilidade na implementação do CSS puro e na migração para o uso do tailwind, além de observar a vantagem do uso de cada um no nosso projeto. Vamos lá?

A seguir temos o código da aula:

Page.css

```
.home{
  display:flex;
  flex-direction:column;
  align-items:center;
  justify-content:center;
  background-color:#1D2123;
  min-height:100vh;
}
.logo{
  color: #ED145B;
  font-family: Arial, sans-serif;
  font-size: 34px;
  justify-content:center;
  align-items:center;
  background-color: inherit;
}
.login-container{
  display:flex;
  flex-direction:column;
```

```

align-items:center;
justify-content:center;
background-color:#1B1E20;
border-radius: 10px;
width: 80vw;
height: 45vh;
margin: 0 auto;
}
.text{
color: #ED145B;
font-family: Arial, sans-serif;
font-size: 24px;
margin-bottom: 10px;
}
.input-login{
width: 70%;
min-height:40px;
padding:5px;
margin:0 auto;
background-color:#1D2123;
border: 1px solid #ED145B;
border-radius:25px;
color: #ED145B;
}
.button{
font-family: Arial, sans-serif;

```

```
font-size: 18px;

background-color: #ED145B;

width: 100px;

border-radius:25px;

margin-top:10px;

}

.button:hover{

  opacity:80%;

}
```

Tailwind.config

```
import type { Config } from "tailwindcss";

const config: Config = {
  content: [
    "./src/pages/**/*.jsx,tsx,mdx",
    "./src/components/**/*.jsx,tsx,mdx",
    "./src/app/**/*.jsx,tsx,mdx",
  ],
  theme: {
    extend: {
      backgroundImage: {
        "gradient-radial": "radial-gradient(var(--tw-gradient-stops))",
        "gradient-conic":
          "conic-gradient(from 180deg at 50% 50%, var(--tw-gradient-stops))",
      },
    },
  },
}
```

```

},
fontFamily: {
  sans: ['Arial', 'sans-serif'],
},
colors: {
  background:{
    800: '#1B1E20',
    900: '#1D2123',
  },
  fontColor:{
    900:'#ED145B',
  }
},
plugins: [],
};
export default config;

```

Page.tsx

```

export default function Home() {
  return (
    <main className='flex flex-1 flex-col h-screen w-screen justify-center items-center bg-background-900'>
      <p className='font-sans text-2xl text-fontColor-900'>FIAPCHAT</p>
      <div className='flex flex-1 flex-col max-h-[45vh] w-[80vw] justify-center items-center bg-background-800'>

```

```

    <p className='font-sans text-1xl text-fontColor-900'>Digite um nome de
usuário</p>

    <input className='font-sans text-lg text-fontColor-900 bg-background-900
border border-fontColor-900 rounded-xl p-2 w-5/6' type="text" placeholder=""/>

    <button className='w-32 mt-2 font-sans bg-fontColor-900 rounded-xl
hover:opacity-80'>Entrar</button>

</div>

</main>

);
}

```

Não deixe de praticar!

SAIBA MAIS

O uso de CSS em Next.js é uma parte fundamental da criação de aplicações web modernas, proporcionando às pessoas desenvolvedoras uma ampla gama de opções para estilizar suas aplicações de forma eficaz e eficiente.

Sendo um framework React, o Next.JS oferece várias abordagens para integrar CSS, cada uma com suas próprias vantagens e casos de uso. Nesta aula, exploraremos as diferentes formas de utilizar CSS em Next.js, destacando práticas recomendadas, vantagens e considerações para otimização de desempenho.

Em sua forma mais básica, o CSS pode ser aplicado em aplicações Next.js da mesma forma que em qualquer outra página HTML ou aplicação JavaScript: arquivos CSS externos podem ser vinculados diretamente a um documento HTML ou importados em componentes JavaScript. No entanto, esta abordagem pode levar a problemas de escopo e manutenção à medida que o projeto cresce.

Styled JSX é outra opção popular dentro do ecossistema Next.js, oferecendo uma forma de escrever CSS diretamente dentro de componentes JavaScript ou TypeScript. Essa abordagem permite que os estilos sejam dinâmicos e dependam do estado ou das props do componente, facilitando a criação de temas ou estilos condicionais. O Styled JSX vem integrado ao Next.js, promovendo uma experiência de desenvolvimento coesa sem a necessidade de configuração adicional.

Para quem prefere utilizar pré-processadores como SASS ou LESS, o Next.js também oferece suporte por meio da instalação de plug-ins ou configurações específicas do webpack. O uso de pré-processadores pode aumentar a produtividade e oferecer funcionalidades avançadas, como variáveis, mixins e funções para manipulação de cores e temas.

A integração de frameworks CSS como Tailwind CSS ou Bootstrap em aplicações Next.js é uma prática comum, proporcionando um conjunto robusto de utilitários e componentes pré-estilizados. Esses frameworks podem acelerar significativamente o desenvolvimento de interfaces de usuário, embora possam também introduzir um excesso de estilos não utilizados, afetando o desempenho. Felizmente, muitos desses frameworks oferecem ferramentas para purificar e otimizar o CSS final, removendo regras não utilizadas.

Usar CSS puro em projetos Next.js, em vez de pré-processadores como Sass/Less ou frameworks CSS-in-JS como styled-components, pode oferecer várias vantagens, dependendo do escopo do projeto, das preferências da equipe e dos objetivos de desempenho.

Aqui estão algumas das principais vantagens de usar CSS puro em Next.js:

- CSS puro é direto e fácil de entender, especialmente para desenvolvedores(as) que estão começando ou para equipes que preferem manter uma abordagem mais tradicional para estilização. Não requer ferramentas adicionais ou etapas de compilação específicas para pré-processadores.
- Usar CSS puro pode resultar em uma carga mais rápida da página, pois não há necessidade de processamento ou conversão em tempo de execução, como é o caso de algumas soluções CSS-in-JS. Menos JavaScript para parsear significa que o tempo até a interatividade (TTI) pode ser reduzido.
- O CSS é padrão em todos os navegadores modernos, garantindo que os estilos sejam consistentes em diferentes ambientes sem a necessidade de preocupações com a compatibilidade de runtime específicas de frameworks CSS-in-JS.

Ao evitar camadas adicionais de abstração, como as introduzidas por pré-processadores ou bibliotecas CSS-in-JS, o pipeline de construção é simplificado. Isso pode tornar o processo de depuração mais direto, já que você está trabalhando mais próximo do resultado que será entregue ao navegador.

Um aspecto fundamental do Next.js é seu sistema de roteamento integrado e fácil de usar. O roteamento é baseado no sistema de arquivos, o que significa que cada arquivo no diretório pages se torna automaticamente uma rota acessível. Este sistema elimina a necessidade de configurar explicitamente rotas, como é comum em muitos outros frameworks.

Por outro lado, o uso do Tailwind CSS em aplicações Next.js representa uma combinação poderosa para o desenvolvimento de interfaces web modernas. Essa parceria aproveita a eficiência e a flexibilidade do Tailwind CSS, um framework CSS utility-first, com a robustez e as funcionalidades avançadas do Next.js, um framework

React para produção. A integração do Tailwind CSS em projetos Next.js oferece aos(as) desenvolvedores(as) um caminho ágil para construir interfaces visualmente atrativas e responsivas, mantendo, ao mesmo tempo, um alto nível de personalização e controle sobre o design final da aplicação.

Veja alguns benefícios, práticas recomendadas e considerações ao utilizar o Tailwind CSS em aplicações Next.js:

- **Agilidade no desenvolvimento e produtividade:** o Tailwind CSS fornece um conjunto extenso de classes utility, que podem ser combinadas para criar designs complexos diretamente no HTML (ou JSX no caso de React/Next.js), eliminando a necessidade de escrever folhas de estilo CSS personalizadas para a maioria dos casos. Isso pode acelerar significativamente o processo de desenvolvimento, permitindo que as pessoas desenvolvedoras construam interfaces com rapidez sem sacrificar a qualidade do design.
- **Responsividade e design adaptável:** Tailwind CSS inclui utilitários para design responsivo, facilitando a implementação de interfaces que se adaptam a diferentes tamanhos de tela. Isso é particularmente vantajoso em Next.js, onde a performance e a otimização para dispositivos móveis são cruciais para o sucesso de uma aplicação web.
- **Consistência e manutenção facilitada:** ao utilizar um conjunto padronizado de classes utility, o Tailwind CSS ajuda a manter a consistência do design em toda a aplicação. Isso simplifica a manutenção e as atualizações do design, uma vez que as alterações podem ser feitas de forma centralizada, ajustando as configurações do Tailwind ou substituindo classes em componentes específicos.
- **Personalização e configuração:** uma das características mais poderosas do Tailwind CSS é sua capacidade de ser altamente personalizado para se adequar à identidade visual de qualquer projeto. Com Next.js, essa personalização é simplificada por meio da configuração do Tailwind, permitindo que os(as) desenvolvedores(as) definam temas, cores, breakpoints e outros valores padrões que serão utilizados em toda a aplicação. Quando um usuário acessa a página, o Next.js verifica se é hora de regenerar a página. Se for, ele serve a versão em cache e regenera a página em segundo plano. Isso

proporciona a velocidade de uma página estática com a flexibilidade de atualizações de conteúdo quase em tempo real.

A integração do Tailwind CSS com Next.js não apenas facilita a criação de interfaces atrativas, mas também contribui para a otimização de desempenho. O Tailwind CSS, com sua abordagem de incluir apenas os estilos utilizados, combinado com a capacidade do Next.js de dividir o código e carregar dinamicamente os componentes, pode reduzir significativamente o tamanho dos pacotes e melhorar os tempos de carregamento das páginas. Essa otimização é crítica para SEO e para proporcionar uma melhor experiência ao usuário, especialmente em conexões lentas ou dispositivos móveis.

A utilização do Tailwind CSS em projetos Next.js promove um fluxo de trabalho ágil e iterativo, permitindo que as pessoas desenvolvedoras façam ajustes de design rapidamente sem a necessidade de refatorar grandes quantidades de CSS personalizado. Isso é particularmente útil em ambientes de desenvolvimento ágeis, onde mudanças rápidas e feedback contínuo são a norma. A capacidade de ajustar a estilização diretamente no markup facilita a experimentação e a iteração rápida, características essenciais para a prototipagem eficaz e para a adaptação ágil às necessidades dos usuários.

Em grandes projetos Next.js, manter a consistência visual em toda a aplicação pode ser desafiador. O Tailwind CSS ajuda a enfrentar esse desafio, oferecendo um sistema de design consistente e facilmente escalável. A personalização do tema do Tailwind permite que os desenvolvedores definam paletas de cores, tamanhos de fonte, espaçamentos e outros valores de design de forma centralizada, garantindo que mudanças no design sejam refletidas em toda a aplicação de maneira uniforme. Isso não só melhora a consistência visual, mas também simplifica o processo de branding e rebranding.

A clareza e a previsibilidade do Tailwind CSS podem facilitar a colaboração entre designers e desenvolvedores(as). Os(as) designers podem especificar layouts e componentes usando o conjunto de utilities do Tailwind, que as pessoas desenvolvedoras podem implementar diretamente, sem a necessidade de traduzir os designs para um conjunto personalizado de estilos CSS. Essa colaboração direta pode reduzir os mal-entendidos e acelerar o desenvolvimento, garantindo que o produto final esteja alinhado com a visão do design.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu como diversas estilizações em projetos Next.js funcionam, explorando as ferramentas e vantagens de algumas delas. Inicialmente, criamos nossa estilização usando CSS puro e, posteriormente, migramos para o Tailwind, abordando as configurações necessárias e seu sistema de estilização baseado em classes.



REFERÊNCIAS

MDN WEB DOCS. **CSS**. [s.d.]. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/CSS> Acesso em: 27 jun. 2024.

NEXT.JS. **Documentação oficial**. [s.d.]. Disponível em: <https://nextjs.org/docs/>. Acesso em: 27 jun. 2024.

TAILWIND CSS. **Documentação oficial**. [s.d.]. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 27 jun. 2024.

EXEMPLO

PALAVRAS-CHAVE

NextJs. CSS. Tailwind.

EMENDAS

POS TECH